

Reverse Engineering & Malware Analysis Tools

The analyst's kit

Tool	Role
Ghidra (NSA)	World-class disassembler + decompiler, GUI, scripts
Radare2 / Cutter	Terminal & GUI reverse engineering framework
GDB + pwndbg/gef	Runtime debugging; pwndbg shines on CTFs binaries
objdump / readelf	Fast static looks at sections/symbols
strings / binwalk	Quick reconnaissance of unknown binaries
YARA	Signature rules to detect malware families
VirusTotal / MALTEGO	Cloud triage (hash check out)
Cuckoo / malware sandboxes	Dynamic analysis in a disposable VM

Static analysis flow

```
file sample.bin
strings sample.bin
readelf -h sample.bin      # ELF headers
objdump -d sample.bin | head # disassembly preview
ghidra sample.bin          # or: r2 -A sample.bin
```

In Ghidra: locate main, look for suspicious imports (socket, exec, VirtualAlloc), decode the strings XOR'd in the binary.

YARA the fast way

```
rule Suspicious_Network
{
  strings:
```

```
$s1 = "cmd.exe" ascii nocase
$s2 = { 6A 00 6A 00 6A 00 } // push 0x0 x3 (socket args)
condition:
    any of them
}
```

```
yara /rule.yar /path/to/samples
```

Dynamic analysis (in a sandbox)

Snapshot the VM, install the sample, watch with procmon (Windows) or strace (Linux)

```
strace -f ./sample
```

 reveals file/network/syscall behavior

Cuckoo auto-reports network/process visual timeline

Malware OPSEC basics

NEVER run samples on your host; a dedicated VM + snapshots only

Disconnect the sandbox network or use a fake connection log

Hash everything (VirusTotal lookup by hashes first – avoid uploading private samples where policy forbids)

The mindset of RE: hypotheses + evidence. Every reverse step answers "what does this binary intend?" – the target of every malware doc.

[← Forensics & Steganography To...](#)

[OSINT & Specialized Tools →](#)